

CSE 256 | Lecture 7: Language modeling + RNNs

Ndapa Nakashole

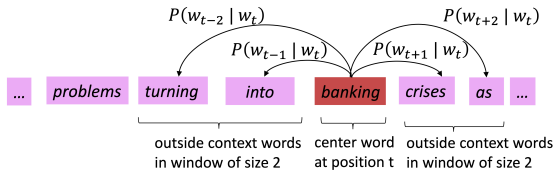
January 27, 2026

Administrative matters

- ▶ **PA1:** due on Friday, January 30
- ▶ **PA2:** out on Monday, February 2
- ▶ **Midterm:** on March 3, here in the classroom

Recap: Word Embeddings, Tokenization

Subword Tokenization:



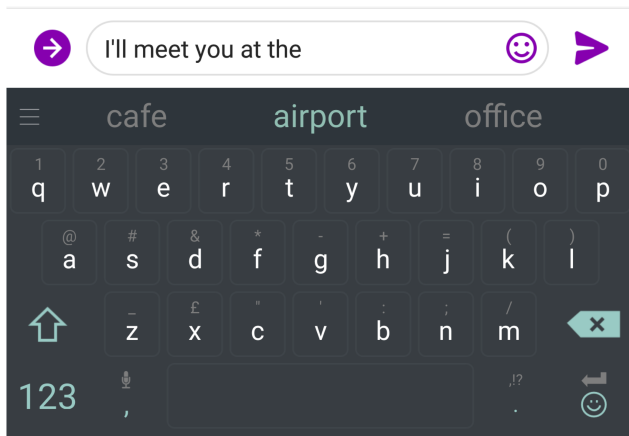
Subword Tokenization:

- ▶ Middle ground between characters and words
- ▶ Reasonable vocabulary size
- ▶ Can represent **unseen words** by composing known subwords

Today

- ① A new NLP task: **Language Modeling**
- ② A new family of neural networks: **Recurrent Neural Networks (RNNs)**

Language Models in Everyday Life



- ▶ Predictive text on your phone (small models)
- ▶ Autocomplete in Google Docs
- ▶ ...

A Language Model: is this sentence OK?

- ① Timo went to the store.
- ② Timo goed to the store.
- ③ The store went to Timo.
- ④ The icecream truck went to Timo
- ⑤ UCSD is located in Texas.

A Language Model: is this sentence OK?

- ① Timo went to the store.
- ② Timo goed to the store. **Grammar**
- ③ The store went to Timo. **Categorical preferences**
- ④ The icecream truck went to Timo. ... **Exceptions**
- ⑤ UCSD is located in Texas. ... **Knowledge about the world**

What Is a Language Model?

A language model learns what sequences of words are *likely*

- ▶ Given what has already been seen
- ▶ e.g., Human infants drink _____

The Language modeling problem

Language modeling is the problem of assigning probabilities to sequences, $P(w_1, \dots, w_T)$, by repeatedly predicting what comes *next*:

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

- ▶ Everything that follows is a design choice about how to represent “what has already happened”

A Naive Language Model

- ▶ We have N training sentences
- ▶ For any sentence $x_1, \dots, x_n$:
- ▶ $c(x_1, \dots, x_n) \leftarrow$ number of times the sentence is seen in our training data

Naive Probability Estimate

$$p(x_1, \dots, x_n) = \frac{c(x_1, \dots, x_n)}{N}$$

This is a valid probability distribution, and a terrible language model: assigns probability zero to any sentence not in the training data

Perplexity Metric For Language Model Quality

We have test data consisting of m sentences: $x_1, \dots, x_m$

- ▶ Probability of $x_{1:m}$ is

$$\prod_{i=1}^m p(x_i)$$

- ▶ Log-probability of $x_{1:m}$ is

$$\sum_{i=1}^m \log_2 p(x_i)$$

- ▶ Average log-probability per word of $x_{1:m}$ is

$$l = \frac{1}{M} \sum_{i=1}^m \log_2 p(x_i)$$

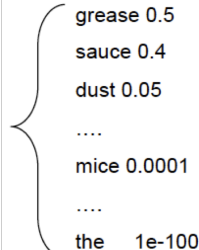
where $M = \sum_{i=1}^m |x_i|$ (total number of words)

- ▶ Perplexity (relative to $x_{1:m}$) is 2^{-l} Lower is better.

Perplexity: How Many Words is the Model Choosing From?

- ▶ The LM assigns probabilities to possible next words. Perplexity measures how *uncertain* the model is about that choice
 - ⊙ confident → one word has most of the probability mass
 - ⊙ uncertain → probability mass is spread over many words

When I eat pizza, I wipe off the _____



grease	0.5
sauce	0.4
dust	0.05
....	
mice	0.0001
....	
the	1e-100

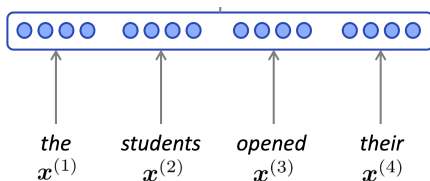
Perplexity is the effective number of choices the model is making.

Neural Language Models

- ① Feedforward neural network LMs
- ② Recurrent neural network (RNN) LMs
- ③ Transformer neural network LMs (next week)

Composing Word Embeddings

- ▶ To predict what comes next, a language model must summarize the context: *what has already happened*.
- ▶ Words are represented as vectors (word embeddings)
 - ◉ Context consists of many words, not just one
 - ◉ We need a way to *combine* word embeddings into a single representation



We will use neural networks to compose word embeddings into a
context vector

Vector Sum Composition

- ▶ Represent a context by summing the embeddings of its words

$$\mathbf{c} = \sum_{i=1}^k \mathbf{e}(x_i)$$

- ▶ The result is a single vector representing the context
- ▶ Word order is ignored (continuous bag-of-words)



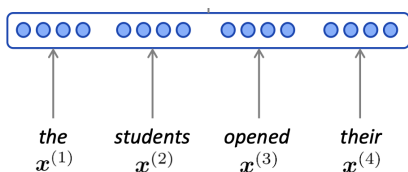
Vector Concatenation Composition

- ▶ A simple way to preserve word order
- ▶ Represent a context by concatenating word embeddings

$$\mathbf{c} = [\mathbf{e}(x_1); \mathbf{e}(x_2); \dots; \mathbf{e}(x_k)]$$

concatenated word embeddings

$$\mathbf{e} = [\mathbf{e}^{(1)}; \mathbf{e}^{(2)}; \mathbf{e}^{(3)}; \mathbf{e}^{(4)}]$$



- ▶ The representation **grows** with the context size 🙄

Fixed-window Neural Language Model

output distribution

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{U}\mathbf{h} + \mathbf{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

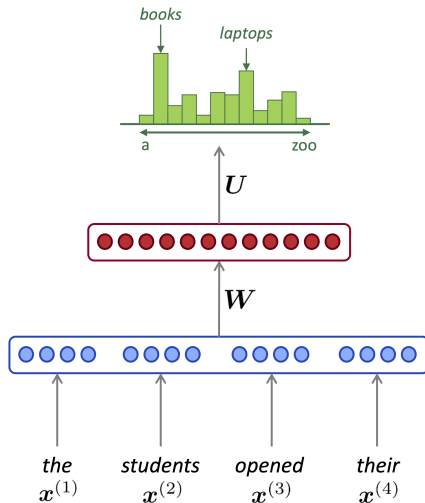
$$\mathbf{h} = f(\mathbf{W}\mathbf{e} + \mathbf{b}_1)$$

concatenated word embeddings

$$\mathbf{e} = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

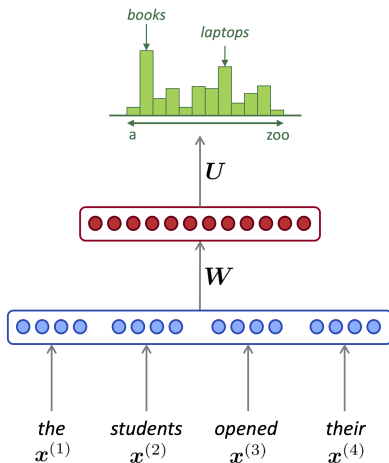
words / one-hot vectors

$$\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}, \mathbf{x}^{(4)}$$



Fixed-window Neural Language Model: Limitations

- ▶ Window size is **too small**
- ▶ Enlarging window enlarges W (parameters)
- ▶ Window can never be large enough!

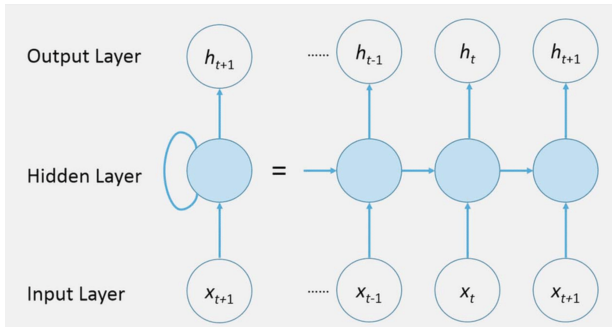


Need a neural architecture that can process any length input

Variable-Length Context

- ▶ We want to condition the next word on *all* preceding words
 - The number of preceding words varies across sentences
 - Variable-length input should not mean a variable number of parameters

Solution: Recurrent neural networks



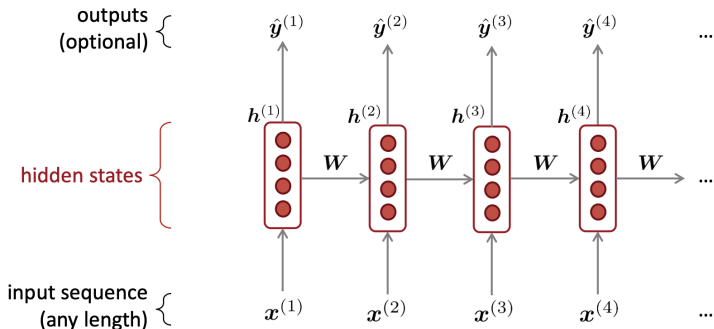
Recurrent Neural Networks (RNNs)

hidden states

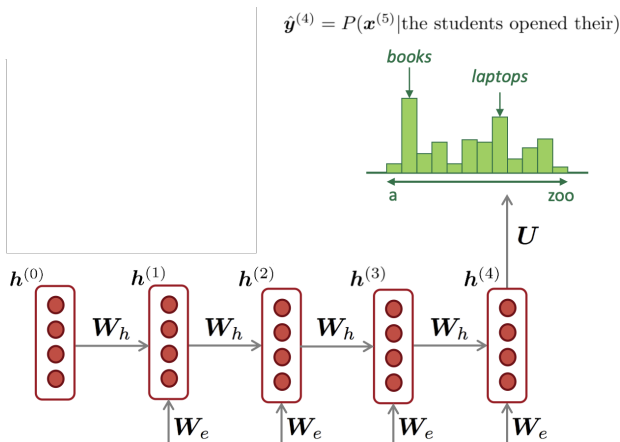
$$h^{(t)} = \sigma \left(W_h h^{(t-1)} + W_e e^{(t)} + b_1 \right)$$

$h^{(0)}$ is the initial hidden state

Core idea: Apply the same weights W repeatedly



RNN Language Model



- ▶ The hidden state summarizes all previous words
- ▶ The model predicts the next word conditioned on the entire history

RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax}(U h^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

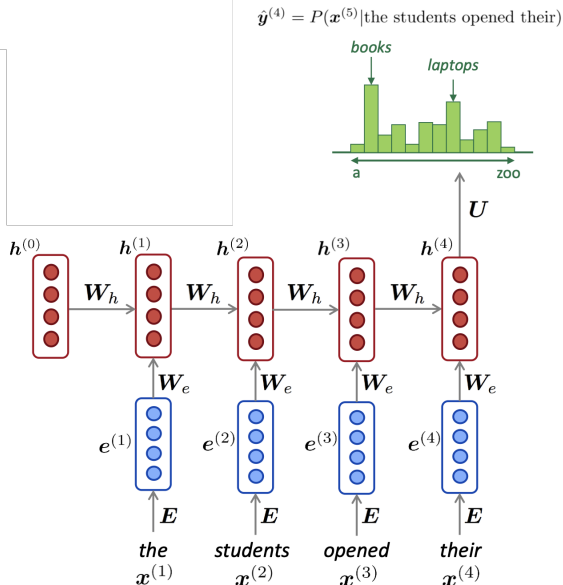
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



Training an RNN Language Model

- ▶ We start with a large corpus of text:

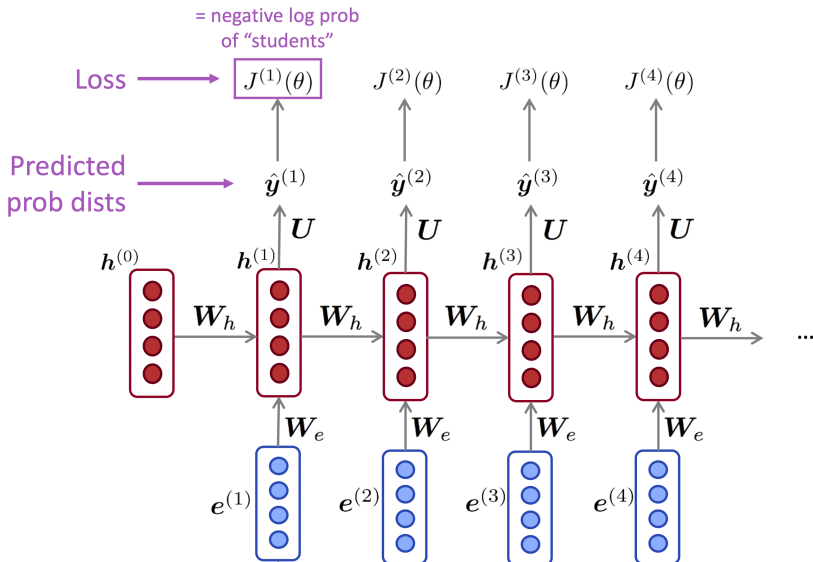
$$\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(T)}$$

- ▶ At each time step t , the model outputs a distribution $\hat{\mathbf{y}}^{(t)}$ over the next word, conditioned on all previous words

The training loss is the average negative log-probability of the true next word:

$$\frac{1}{T} \sum_{t=1}^T -\log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$

Training an RNN Language Model



RNNs greatly improved perplexity over previous methods

n-gram model →

Increasingly complex RNNs ↓

Model	Perplexity
Interpolated Kneser-Ney 5-gram (Chelba et al., 2013)	67.6
RNN-1024 + MaxEnt 9-gram (Chelba et al., 2013)	51.3
RNN-2048 + BlackOut sampling (Ji et al., 2015)	68.3
Sparse Non-negative Matrix factorization (Shazeer et al., 2015)	52.9
LSTM-2048 (Jozefowicz et al., 2016)	43.7
2-layer LSTM-8192 (Jozefowicz et al., 2016)	30
Ours small (LSTM-2048)	43.9
Ours large (2-layer LSTM-2048)	39.8

Perplexity improves (lower is better)

Source: <https://research.fb.com/building-an-efficient-neural-language-model-over-a-billion-words/>

RNN Language Models: the Good and the Bad

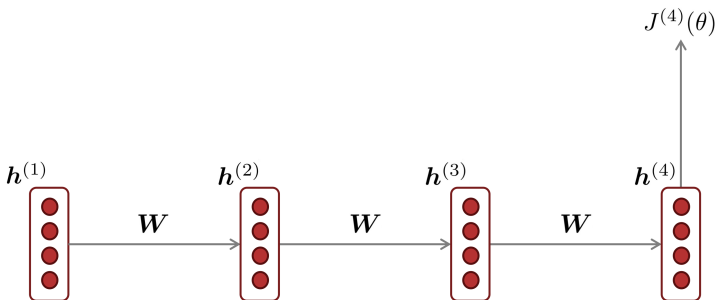
RNN Advantages

- ▶ Can process input of **any length**
- ▶ Computation at step t can (in theory) use information from **many** steps back
- ▶ Model size **does not increase** for longer input

RNN Disadvantages

- ▶ Recurrent **computation is slow**
- ▶ In practice, it is difficult to access information from many steps back

Problems with RNNs

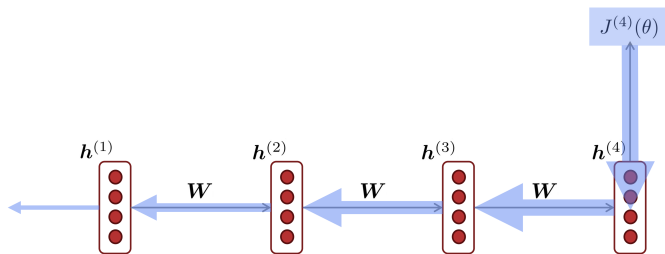


- ▶ Unrolling an RNN through time, creates a very deep feedforward network
 - ⊙ This leads to **very long** chains of multiplication during backpropagation
 - ⊙ Can cause **gradients to vanish or explode**

Key Difference from Feedforward Networks

- ▶ In a feedforward network, each layer has its own parameters

Vanishing Gradients: Intuition

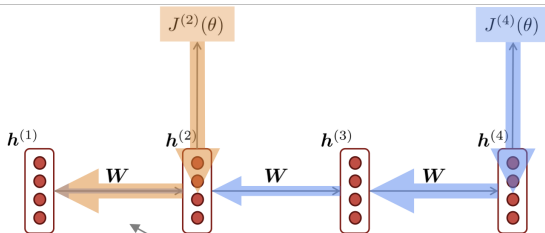


$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

What happens if these are small?

Vanishing gradient problem:
When these are small, the gradient signal gets smaller and smaller as it backpropagates further

Vanishing Gradients: Intuition



Gradient signal from far away is lost because it's much smaller than gradient signal from close-by.

So, model weights are updated only with respect to near effects, not long-term effects.

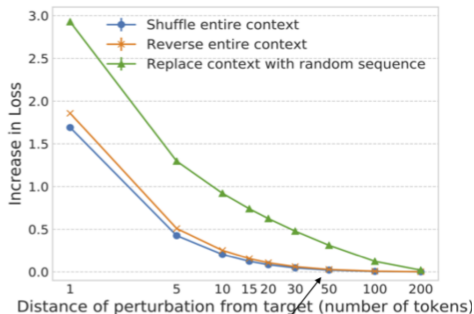
Effect of Vanishing Gradients on an RNN Language Model

- ▶ Consider the language modeling task:

When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____ .

- ▶ To learn from this example, the RNN must connect the word *tickets* early in the sentence to the final prediction
- ▶ This requires a learning signal that reaches back many steps
- ▶ If **the gradient is very small**, the model cannot learn this dependency
- ▶ As a result, the **model struggles with similar long-distance dependencies at test time**

Effect of Vanishing Gradients



History farther than 50 words
away treated as a bag of words.

- ▶ Experimental idea (Khandelwal et al., 2018): progressively shuffle or replace context words farther than k tokens from the target
- ▶ Performance degrades mainly when recent context is disrupted

What About Exploding Gradients?

If the gradient becomes very large, the SGD update step becomes very large:

$$\theta^{\text{new}} = \theta^{\text{old}} - \underbrace{\alpha}_{\text{learning rate}} \underbrace{\nabla_{\theta} J(\theta)}_{\text{gradient}}$$

- ▶ Large gradients lead to **overly large parameter updates**
- ▶ The model can jump to a poor region of parameter space
- ▶ Training becomes unstable and the loss may increase dramatically

In the worst case, numerical values overflow, resulting in **Inf** or **NaN**, and training must be restarted.

Gradient Clipping: A Solution for Exploding Gradients

- ▶ Gradient clipping limits the size of the update without changing its direction

Norm-based gradient clipping

Algorithm 1 Pseudo-code for norm clipping

$$\begin{aligned} \hat{\mathbf{g}} &\leftarrow \frac{\partial \mathcal{E}}{\partial \theta} \\ \text{if } \|\hat{\mathbf{g}}\| &\geq \text{threshold} \text{ then} \\ &\quad \hat{\mathbf{g}} \leftarrow \frac{\text{threshold}}{\|\hat{\mathbf{g}}\|} \hat{\mathbf{g}} \\ \text{end if} \end{aligned}$$

- ▶ **Intuition:** take a step in the same direction, but a smaller step
- ▶ In practice, clipping is simple and effective
- ▶ Exploding gradients are usually easier to handle than vanishing gradients

How Do We Fix the Vanishing Gradient Problem?

- ▶ Main difficulty is learning to preserve information over many steps
- ▶ In a vanilla RNN, the hidden state is repeatedly rewritten

$$\mathbf{h}^{(t)} = \sigma\left(\mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_x \mathbf{x}^{(t)} + \mathbf{b}\right)$$

- ◉ Idea 1: add a separate memory that is updated more gently (LSTM, this lecture)
- ◉ Idea 2: create more direct paths for information to flow (Transformers, next week)

RNNs are General Sequence Encoders

► Represent text sequences

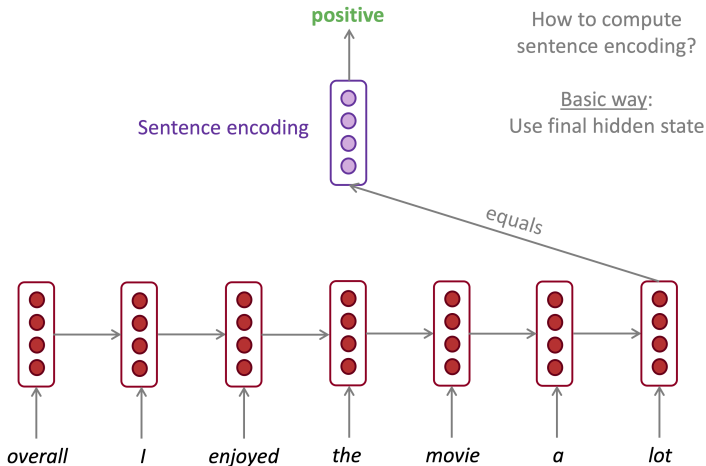
- ◉ Language modeling
- ◉ Text Classification
- ◉ Sequence labeling (e.g., POS tagging, NER)

► More generally

- ◉ RNNs provide a neural architecture for encoding sequential data
- ◉ Time series, and biological sequences (e.g., DNA)

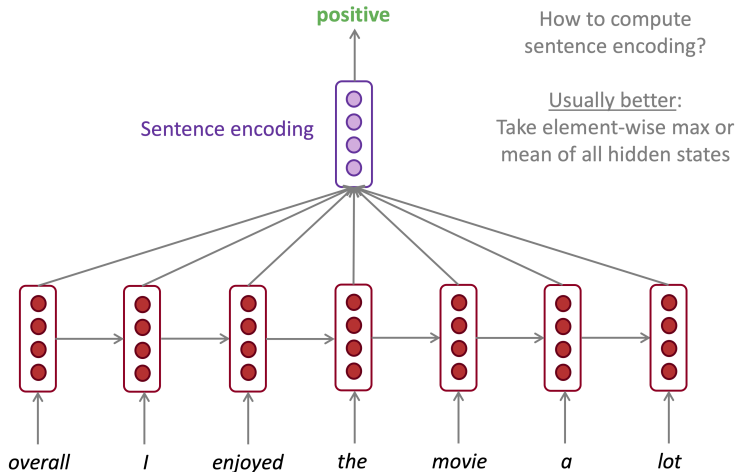
RNNs Sentence Encoding for Text Classification: Use last hidden state

e.g. [sentiment classification](#)



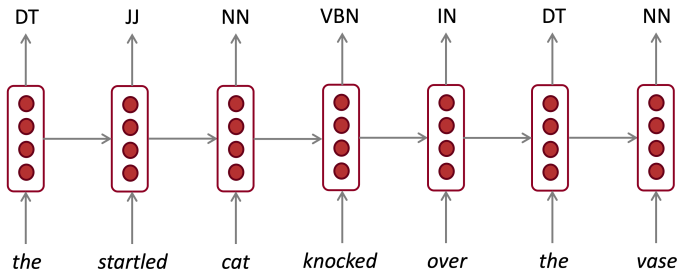
RNNs Sentence Encoding for Text Classification: Sum up all hidden states

e.g. sentiment classification



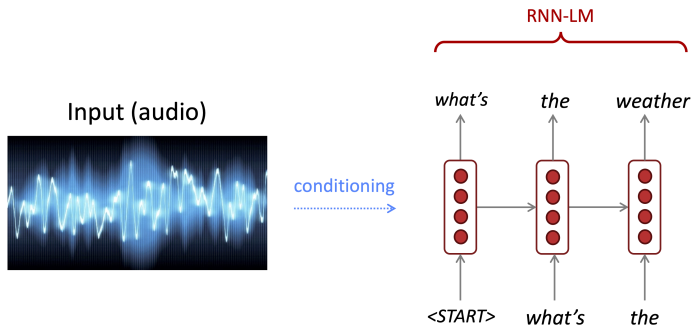
RNNs Applied to Sequence Labeling

e.g. part-of-speech tagging, named entity recognition



RNN-LMs applied to Conditional Generation

e.g., **speech recognition**, machine translation, summarization

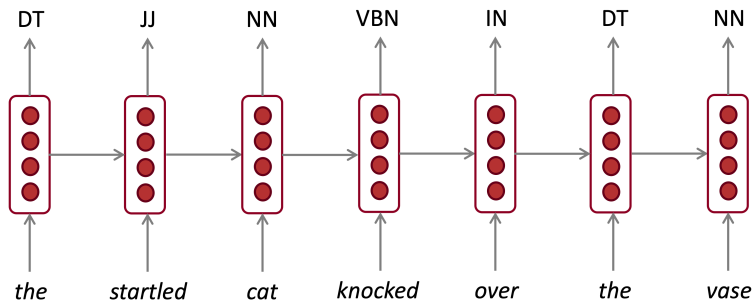


This is an example of a *conditional language model*.
We'll see Machine Translation in much more detail starting next lecture.

The RNN Variants

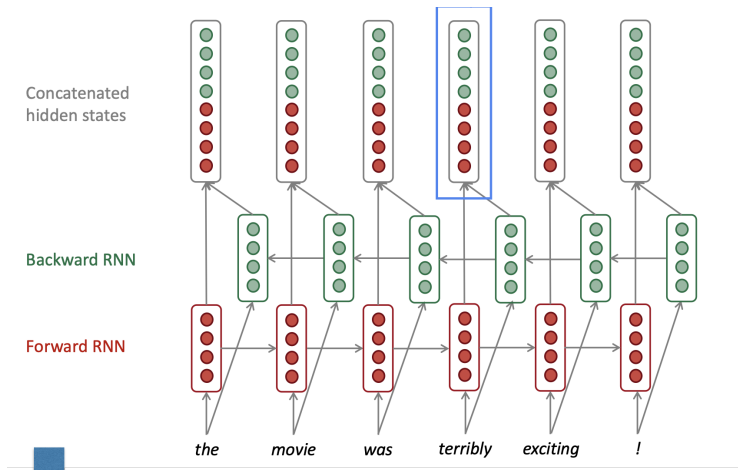
- ▶ Bidirectional RNNs
- ▶ Multi-layer (deep) RNNs
- ▶ Gated RNNs (LSTMs, GRUs)

Unidirectional RNNs: Limitation



- ▶ Information flows in only one direction
- ▶ The prediction at y_4 cannot depend on later inputs $x_5, x_6, \dots, x_n$
- ▶ This is restrictive for tasks where future context matters

Solution: Bidirectional RNNs



- Two RNNs process the same sequence: one runs forward, the other runs backward

Bidirectional RNNs: Notation in the Literature

Forward RNN $\vec{h}^{(t)} = \text{RNN}_{\text{FW}}(\vec{h}^{(t-1)}, \mathbf{x}^{(t)})$

Backward RNN $\overleftarrow{h}^{(t)} = \text{RNN}_{\text{BW}}(\overleftarrow{h}^{(t+1)}, \mathbf{x}^{(t)})$

Concatenated hidden states $\mathbf{h}^{(t)} = [\vec{h}^{(t)}; \overleftarrow{h}^{(t)}]$

Generally, these two RNNs have separate weights

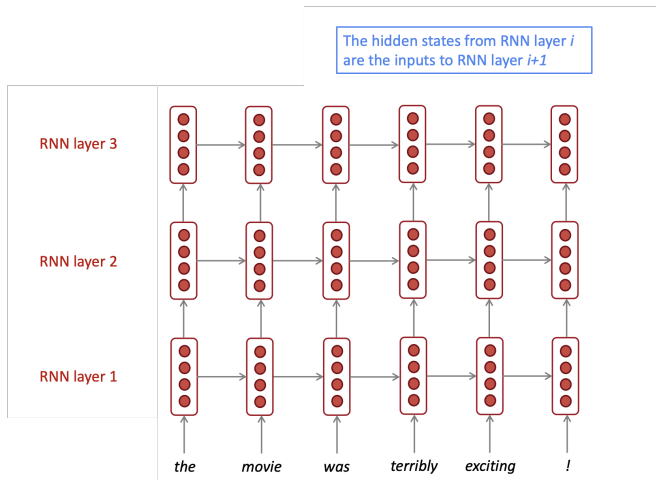
We regard this as “the hidden state” of a bidirectional RNN. This is what we pass on to the next parts of the network.

- ▶ Forward and backward hidden states are computed separately
- ▶ They are typically concatenated at each position

Bidirectionality is Great but ...

- ▶ Bidirectional RNNs require access to the *entire input sequence*
- ▶ They are **not** applicable to language modeling
 - ◉ In language modeling, only left context is available at prediction time
- ▶ Modern sequence encoders build heavily on this idea
 - ◉ **BERT** uses bidirectional context to learn strong sentence representations (later in the course ...)
 - ◉ **Transformers are also bidirectional**

Deep (Multi-layer) RNNs



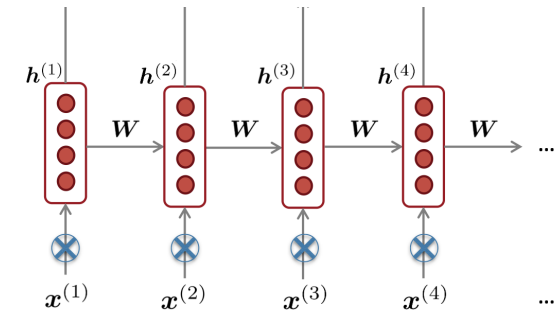
- ▶ RNNs can be stacked to form multiple layers
- ▶ The hidden states from layer i are the inputs to layer $i + 1$

Gated RNNs: LSTMs

Long Short-Term Memory (LSTM) Networks

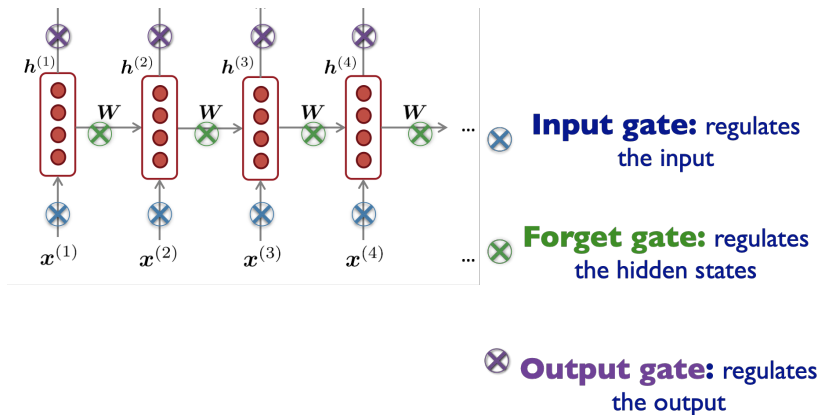
Vanilla RNNs have trouble capturing long-range dependencies

LSTMs: Have special **gated structure** to control memorization and forgetting in RNNs.



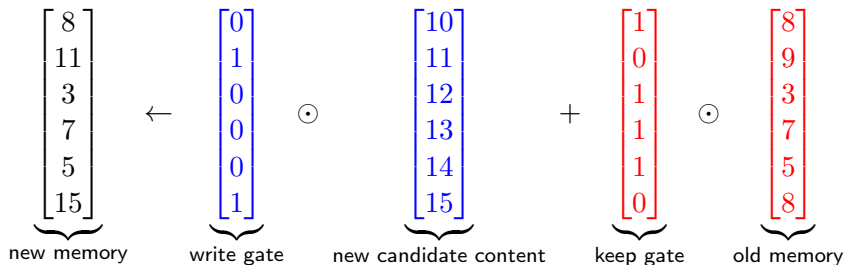
Input gate: Controls what parts of the input make it into the hidden state

LSTM: Input, Forget, and Output Gates



Gate behavior: Hard Gates

- ▶ Binary vectors can act as a 'gate' for controlling access to vectors, using element-wise products
- ▶ Gates are closed (0) or open (1)



Gate Behavior: Computed Soft Gates

- ▶ Want soft, differentiable gates
- ▶ Gate values lie between 0 (closed) and 1 (open)
- ▶ Gates are computed from the current input and previous hidden state

Sigmoid function: ensures all gate values are in $[0, 1]$

$$\mathbf{f}^{(t)} = \sigma\left(\mathbf{W}_f \mathbf{h}^{(t-1)} + \mathbf{U}_f \mathbf{x}^{(t)} + \mathbf{b}_f\right)$$

$$\mathbf{i}^{(t)} = \sigma\left(\mathbf{W}_i \mathbf{h}^{(t-1)} + \mathbf{U}_i \mathbf{x}^{(t)} + \mathbf{b}_i\right)$$

$$\mathbf{o}^{(t)} = \sigma\left(\mathbf{W}_o \mathbf{h}^{(t-1)} + \mathbf{U}_o \mathbf{x}^{(t)} + \mathbf{b}_o\right)$$

LSTM: Hidden State, Cell State, and Gates

- ▶ At time step t , an LSTM maintains two states:
 - ◉ the hidden state $\mathbf{h}^{(t)}$
 - ◉ the cell state $\mathbf{c}^{(t)}$
- ▶ The hidden state plays a role similar to the vanilla RNN hidden state
- ▶ The cell state stores longer-term information
- ▶ Information in the cell can be erased, written, or read
 - ◉ These operations are controlled by gates
 - ◉ Each gate is a vector of length n with values between 0 and 1

LSTM Update Equations

$$\tilde{\mathbf{c}}^{(t)} = \tanh\left(\mathbf{W}_c \mathbf{h}^{(t-1)} + \mathbf{U}_c \mathbf{x}^{(t)} + \mathbf{b}_c\right) \quad (\text{candidate new cell content})$$

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \circ \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \circ \tilde{\mathbf{c}}^{(t)} \quad (\text{updated cell state})$$

$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \circ \tanh\left(\mathbf{c}^{(t)}\right) \quad (\text{hidden state})$$

- ▶ $\mathbf{c}^{(t-1)}$: previous cell state (long-term memory)
- ▶ $\mathbf{c}^{(t)}$: current cell state after forgetting and writing

Why Gating Helps

- ▶ Gates prevent values from shrinking or growing too much
 - ◉ $\tanh(\cdot)$ produces values between -1 and 1
 - ◉ This stabilizes the hidden and cell states
 - ◉ More stable activations lead to more stable gradients

$$\tilde{\mathbf{c}}^{(t)} = \tanh\left(\mathbf{W}_c \mathbf{h}^{(t-1)} + \mathbf{U}_c \mathbf{x}^{(t)} + \mathbf{b}_c\right) \quad (\text{candidate new cell content})$$

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \circ \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \circ \tilde{\mathbf{c}}^{(t)} \quad (\text{updated cell state})$$

$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \circ \tanh\left(\mathbf{c}^{(t)}\right) \quad (\text{visible hidden state})$$

LSTM: Real-World Success

- ▶ **2013–2015:** LSTMs achieved state-of-the-art results across many tasks. Successful applications included:
 - ⊙ handwriting and speech recognition
 - ⊙ machine translation and parsing
 - ⊙ image captioning
 - ⊙ language modeling
- ▶ **2019–:** Transformers became dominant across essentially all tasks

Questions ?